

COM 316
Artificial Intelligence
Scheme 1

Muhammad Abdullah
September 2nd 2026

```
> "hello"
"hello"
> 42
42
> 3.81
3.81
> +
#<procedure +>
> (+ 5 6)
11
> '(a b c d)
(a b c d)

> (define lst '(a b c d))
> (car lst)
a
> (cdr lst)
(b c d)
> (cons 'x lst)
(x a b c d)
```

define is_big
(is_big x) => #t if x is greater than 1000, otherwise #f

```
> (define (is_big x)
  (if (> x 1000)
      #t
      #f
  ))
> (is_big 10001)
#t
> (is_big 101)
#f
```

```
> (is_big 1000)
#f
> (is_big 1001)
#t
```

define sqr:
(sqr x) => x squared

```
> (define (sqr x)
  (* x x))
> (sqr 5)
25
> (sqr 1.77245)
3.1415790025000003
>
```

```
> (define sqr
  (lambda (x)
    (* x x)))
> (sqr 5)
25
> (sqr 4)
16
>
```

define double:
(double x) => 2 * x

```
> (define double
  (lambda (x)
    (+ x x)))
> (double 2026)
4052
```

define dist:
(dist x1 y1 x2 y2) => Euclidean distance from point (x1, y1) to (x2, y2).

```
> (define (dist x1 y1 x2 y2)
  (sqrt (+ (sqr (- x1 x2)) (sqr (- y1 y2)))))
> (dist 10 10 12 12)
2.8284271247461903
```

```
> (dist 10 10 20 30)
22.360679774997898
>
```

type at the prompt:

```
(define pointa '(1 2))
(define pointb '(3 5))
```

define dist_pt:

```
(dist_pt p1 p2)
```

```
> (define dist_pt
  (lambda (p1 p2)
    (sqrt (+ (sqr (- (list-ref p1 0) (list-ref p2 0)))
              (sqr (- (list-ref p1 1) (list-ref p2 1)))))))
> (define pointa '(1 2))
> (define pointb '(3 5))
> (dist_pt pointa pointb)
3.605551275463989
> (define pointa '(10 10))
> (define pointb '(12 12))
> (dist_pt pointa pointb)
2.8284271247461903
> (define pointa '(10 10))
> (define pointb '(20 30))
> (dist_pt pointa pointb)
22.360679774997898
>
```

define find_tf:

(find_tf x lst) => finds if x is in the list lst. it returns #t or #f

```
> (define (find_tf x lst)
  (cond ((null? lst) #f)
        ((equal? x (car lst)) #t)
        (else (find_tf x (cdr lst)))))
> (find_tf 'a '(a b c d e f))
#t
> (find_tf 'z '(a b c d e f))
#f
> (find_tf 'd '(a b c d e f))
#t
>
```

define find_posit:

(find_posit x lst) => returns index of x in the list lst. it returns -1 if not in list

```
> (define (find_posit x lst)
  (define (helper l idx)
    (cond ((null? l) -1)
          ((equal? x (car l)) idx)
          (else (helper (cdr l) (+ idx 1)))))
  (helper lst 0))
> (find_posit 'd '(a b c d e f))
3
> (find_posit 'z '(a b c d e f))
-1
> (find_posit 'a '(a b c d e f))
0
>
```

define count_all:

(count_all lst) => returns the number of elements in the list lst

```
> (define (count_all lst)
  (define (help l count)
    (cond ((null? l) count)
          (else (help (cdr l) (+ count 1)))))
  (help lst 0))
> (count_all '(a b c d e f))
6
> (count_all '(a b (a b e) 3 4))
5
>
```

define count_twos

(count_twos lst) => returns the number of 2s in the list lst

```
> (define count_twos
  (lambda (lst)
    (cond
      ((null? lst) 0)
      ((equal? (car lst) 2) (+ 1 (count_twos (cdr lst))))
      (else (count_twos (cdr lst)))))
> (count_twos '(1 2 2 2 3 4 2 3))
4
```

```
> (count_twos '(1 0 0 0 0 3 4 3))  
0  
> (count_twos '(1 0 2 20 0 3 4 3))  
1
```